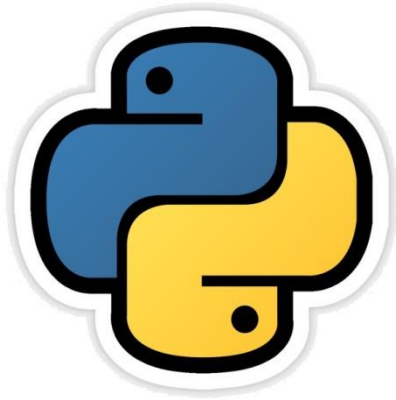




Functions

सीबीएसई पाठ्यक्रम पर आधारित
कक्षा -12

अध्याय -2



द्वारा:

संजीव भदौरिया

स्नातकोत्तर शिक्षक (संगणक विज्ञान)

के० वि० बाराबंकी (लखनऊ संभाग)

Functions in Python

- एक Function statements का एक ऐसा समूह होता है जो प्रोग्राम में किसी एक निश्चित कार्य को करने के लिए बनाया जाता है ।
- इसे execute करने के लिए इसको कॉल करना आवश्यक है ।
- एक बहुत बड़ा प्रोग्राम न लिख कर हम उसके छोटे छोटे हिस्से करके function बनाकर उनका प्रयोग कर सकते हैं । जिसमे से हर हिस्सा एक निश्चित कार्य के लिए बनाया हुआ होगा ।
- एक बार बनाये गए function को हम दूसरे प्रोग्राम में भी प्रयोग कर सकते हैं ।
- Functions को 3 प्रकार से वर्गीकृत किया जा सकता है -
 1. Built-in
 2. Modules
 3. User Defined

Built-in Functions

- ये वह function होते हैं जो python में पहले से बनाये गए हैं ।
- Functions किसी भी प्रोग्रामिंग भाषा को दक्ष (Efficient) बनाते हैं और उसे structure प्रदान करते हैं ।
- Python में कई built-in functions हैं जो प्रोग्रामिंग को आसान, तेज़ और पावरफुल बनाते हैं ।
- ये हमेशा standard library में रहते हैं और इनके लिए हमें module को इम्पोर्ट करने की आवश्यकता नहीं होती है ।
- कुछ built-in function के बारे में हम अगली स्लाइड में पढ़ेंगे ।

Built-in Functions. . .

1. Type Conversion Functions: ये वह function होते हैं जो मानों (values) को एक प्रकार से दूसरे प्रकार में बदलते हैं -

1. `int()` – स्ट्रिंग को integer में बदलने के लिए
2. `str()` – किसी भी वैल्यू को स्ट्रिंग में बदलने के लिए
3. `float()` – स्ट्रिंग को float में बदलने के लिए

2. Input Functions: यह यूजर से स्ट्रिंग को इनपुट लेने के काम आता है | जिसका अध्ययन हम कक्षा -11 में कर चुके हैं |

e.g. `name=input("Enter your name : ")`

3. eval function: यह function स्ट्रिंग की वैल्यू को evaluate करने के काम आता है |

e.g. `x=input("45+10")`

`print(x)`

`# answer will be 55`

Built-in Functions. . .

3. min Function: यह दी गयी values में से सबसे छोटी वैल्यू को return करता है ।

```
>>> min(12, 5, 7, -102)
-102
```

```
>>> max(12, 5, 7, -102)
12
```

4. max Function: यह दी गयी values में से सबसे बड़ी वैल्यू को return करता है ।

```
>>> abs(-22)
22
```

```
>>> abs(22)
22
```

5. abs Function: यह एक संख्या की absolute value को return करता है जो की सकारात्मक (+ve) संख्या होती है ।

Built-in Functions. . .

6. type Function: यदि आप किसी variable का प्रकार (type) पता करना चाहते हैं तो यह function काम आता है |

7. len Function: यह function किसी स्ट्रिंग की लम्बाई को व्यक्त करता है अर्थात उसमें कितने कैरेक्टर हैं यह return करता है |

8. round Function: यह function किसी संख्या को दिए गए संख्या तक round करने के काम आता है |

```
>>> type(10)
<class 'int'>
>>> a="hello"
>>> type(a)
<class 'str'>
>>> b=20
>>> type(b)
<class 'int'>
>>> len(a)
5
>>> round(11.12345,2)
11.12
>>> round(11.12345,3)
11.123
>>> round(11.12345,0)
11.0
```

Built-in Functions. . .

9. range Function: यदि आप किन्हीं दो संख्याओं के बीच की निश्चित अंतराल के बाद संख्याओं की series चाहते हैं तो range function इसमें काफी मददगार होता है | यह for लूप के लिए अत्यंत सहायक टूल होता है | इसका syntax निम्न है -

range(start, stop, step)

यह start से शुरू होकर stop – 1 तक की वैल्यू को return करता है वह भी step के अंतराल के बाद

```
>>> list(range(0,5))  
[0, 1, 2, 3, 4]
```

```
for i in range(1,10):  
    print(i)
```

= RESTART:

1
2
3
4
5
6
7
8
9

Python Modules

- Module एक फाइल होती है जिसमे functions और variables परिभाषित किये रहते हैं ।
- एक module पाइथन कोड की एक साधारण फाइल होती है ।
- जब हम किसी प्रोग्राम को modules में विभक्त करते हैं तो हर module में functions और variables होते हैं जहाँ हर function किसी विशेष कार्य के लिए बनाया गया होता है ।
- Module के द्वारा एक बार लिखा गया कोड किसी दूसरे प्रोग्राम में भी प्रयोग में लाया जा सकता है ।
- जब हम किसी एक या अधिक ऐसे function बनाते हैं जिन्हें हमें दूसरे प्रोग्राम में भी प्रयोग करना है, तो हम in functions को module में लिखते हैं ।
- फिर in module को हम जिस प्रोग्राम में चाहें import करके functions को प्रयोग कर सकते हैं।

Python Modules. . .

- पाइथन module को import करने के लिए 2 विधियाँ प्रदान करता है -
- **import स्टेटमेंट** : पूरे module को import करने के लिए
- **from:** सारे या चुनिन्दा functions को import करने के लिए

```
import math  
a=20  
print(math.sqrt(a))
```

```
from math import sqrt  
a=20  
print(sqrt(a))
```

4.47213595499958

Python Modules. . .

- ऐसा करने से सबसे पहले python द्वारा math.py फाइल खोजी जाती है ।
- फिर space बनाई जाती है जहाँ module के definition और variables को रखा जा सके ।
- फिर module के statements को execute किया जाता है ।

```
import math  
a=20  
print(math.sqrt(a))
```

4.47213595499958

math Module

- math module में निम्न functions होते हैं –

- **ceil(x)** यह x से बड़ा या बराबर का integer return करता है।
- **floor(x)** यह x से छोटा या बराबर का integer return करता है।
- **pow(x, n)** यह x^n की वैल्यू return करता है।
- **sqrt(x)** यह x का वर्गमूल (square root) return करता है।
- **log10(x)** यह base-10 का logarithm return करता है।
- **cos(x)** यह x का cosine वह भी radians में return करता है।
- **sin(x)** यह x का sine वह भी radians में return करता है।
- **tan(x)** यह x का tangent वह भी radians में return करता है।

```
>>> import math
>>> math.ceil(5.6)
6
>>> math.floor(-43.7)
-44
>>> math.floor(43.7)
43
>>> math.pow(3, 4)
81.0
>>> math.sqrt(25)
5.0
>>> math.log10(100)
2.0
>>> math.cos(60)
-0.9524129804151563
>>> math.cos(0)
1.0
>>> math.sin(45)
0.8509035245341184
>>> math.sin(0)
0.0
>>> math.tan(45)
1.6197751905438615
>>> math.tan(90)
-1.995200412208242
```

help Function

- यदि आप यह भूल गए हैं कि कोई library function कैसे काम करता है तो help library function आपकी सहायता करता है। तथा यह किसी भी module के साथ प्रयोग की जाए तो यह उस module के अन्दर समस्त function और उनकी जानकारी भी देता है।

```
>>> import math
```

```
>>> print(help(math.cos))
```

```
Help on built-in function cos in module math:
```

```
cos(...)
```

```
cos(x)
```

```
Return the cosine of x (measured in radians).
```

```
None
```

string Module

- math module के बारे में हम कक्षा 11 में पढ़ ही चुके हैं जिसके साथ निम्न के अलावा भी अनेक functions होते हैं –

- [String.capitalize\(\)](#) Converts first character to Capital Letter
- [String.find\(\)](#) Returns the Lowest Index of Substring
- [String.index\(\)](#) Returns Index of Substring
- [String.isalnum\(\)](#) Checks Alphanumeric Character
- [String.isalpha\(\)](#) Checks if All Characters are Alphabets
- [String.isdigit\(\)](#) Checks Digit Characters
- [String.islower\(\)](#) Checks if all Alphabets in a String, are Lowercase
- [String.isupper\(\)](#) returns if all characters are uppercase characters
- [String.join\(\)](#) Returns a Concatenated String
- [String.lower\(\)](#) returns lowercased string
- [String.upper\(\)](#) returns uppercased string
- [len\(\)](#) Returns Length of an Object
- [ord\(\)](#) returns Unicode code point for Unicode character
- [reversed\(\)](#) returns reversed iterator of a sequence
- [slice\(\)](#) creates a slice object specified by range()

random Module

- जब हमें कुछ ऐसी संख्या चाहिए होती है जहाँ संख्या पहले से पता न हो जैसे की captcha या किसी सीरियल नंबर को बनाना तो ऐसे समय random संख्याओं की ज़रूरत होती है और इसके लिए random module सहायता करता है | इसमें निम्न functions होते हैं -
- randrange ()**: यह method इसके साथ दिए गए lower और upper argument के मध्य कोई भी integer return करता है | default lower वैल्यू जीरो (0) और upper वैल्यू वन (1) होती है |

```
>>> import random
>>> random.randrange(15)
0
>>> random.randrange(15)
10
```

```
import random
sub=["CS", "IP", "Phy", "Chem", "Maths"]
ran_index=random.randrange(3)
print(sub[ran_index])
```

```
= RESTART:
IP
>>> |
```

```
>>> import random
>>> random.random()
0.9576497696403067
>>> random.random()
0.05005384317166639
```

random Module. . .

- **randint ()**: यह method 2 पैरामीटर लेता है a,b जिसमे पहला lower तथा दूसरा upper limit होती है | यह इन दोनों संख्याओं के मध्य कोई भी संख्या return कर सकती है जिसमे दोनों limit भी शामिल हैं | यह method, guessing applications के लिए सबसे सही होता हैं |

```
#Program to generate any two numbers between 0 to 10
```

```
import random
print(random.randint(0,10))
print(random.randint(0,10))
```

```
= RESTART:
8
4
```

- **uniform ()** यह method दो संख्याओं के बीच की कोई भी floating-point संख्या को return करता है |

```
>>> import random
>>> print("Uniform Lottery Number (1,100): ",random.uniform(1,100))
Uniform Lottery Number (1,100): 85.66278453560943
```

random Module. . .

- **choice ()**: यह method किसी list, tuple या string से random selection के लिए प्रयोग किया जाता है।

```
#Program to select My food from a list
```

```
import random
lst=["Daal", "Chaaval", "Roti", "MixVeg", "Achar", "PaneerMasala", "malaiKofta"]
print("My First item is : ",random.choice(lst))
print("My Second item is : ",random.choice(lst))
```

= RESTART: C:/Users/KVBBKServer
My First item is : malaiKofta
My Second item is : MixVeg

- **shuffle ()** यह method किसी list के contents को shuffle या swap कर सकता है।

```
#Program to shuffle My food list
```

```
import random
lst=["Daal", "Chaaval", "Roti", "MixVeg", "Achar", "PaneerMasala", "malaiKofta"]
random.shuffle(lst)
print("My shuffled list is : ",lst)
random.shuffle(lst)
print("My Second time shuffled list is : ",lst)
```

```
= RESTART: C:/Users/KVBBKServer/AppData/Local/Programs/Python/Python36/a.py =
My shuffled list is : ['Achar', 'Daal', 'Chaaval', 'malaiKofta', 'PaneerMasala', 'Roti', 'MixVeg']
My Second time shuffled list is : ['Chaaval', 'PaneerMasala', 'Daal', 'MixVeg', 'Roti', 'Achar', 'malaiKofta']
```

User-defined Functions

- ये वह function होते हैं जो user के द्वारा बनाये जाते हैं जिनको यूजर अपने अनुसार प्रयोग कर सकता है ।
- Function एक प्रकार का statements का समूह होता है जो किसी निश्चित कार्य को करने के लिए बनाया जाता है ।
- इसको हम अपने पाइथन के प्रोग्राम के किसी भी हिस्से में कितनी भी बार कॉल करके प्रयोग कर सकते हैं ।
- User-defined function बनाने के लिए def keyword का प्रयोग किया जाता है ।

User-defined Functions. . .

- User-defined function बनाने के लिए def keyword के साथ निम्न syntax का प्रयोग किया जाता है |

def Function_Name(List_Of_Parameters):

"""docstring"""

statement(s)

Keyword

def की लाइन बाद वाली लाइन को 4 space के बाद लिखा जाना चाहिए जिसे indentation कहते हैं | और यही function का block या body कहलायेगा |

Function Definition

User-defined Functions **without argument** and **without return** (Practical)

#Program to add two numbers using function without argument

```
def add():  
    a=int(input("Enter value of a : " ))  
    b=int(input("Enter value of b : " ))  
    c=a+b  
    print("The sum is : ",c)
```

Function
Definition

add()

Function Call

```
= RESTART: C:/Users/KV  
Enter value of a : 12  
Enter value of b : 23  
The sum is : 35
```

User-defined Functions **with argument** and **without return** (Practical)

#Program to add two numbers using function with argument

```
def add(a,b):  
    c=a+b  
    print("The sum is : ",c)  
  
x=int(input("Enter value of a : " ))  
y=int(input("Enter value of b : " ))  
add(x,y)
```



Argument के केस में variables में इनपुट function के बहार लिया जाता है और उन variables को अरगुमेंट के रूप में calling स्टेटमेंट में पास किया जाता है

```
= RESTART: C:/Users/KV  
Enter value of a : 12  
Enter value of b : 23  
The sum is : 35
```

User-defined Functions **with argument** and **with return value** (Practical)

```
'''Program to add two numbers using function
with argument and return statement
'''
def add(a,b):
    c=a+b
    return c

x=int(input("Enter value of a : " ))
y=int(input("Enter value of b : " ))
z=add(x,y)
print("The sum is : ",z)
```



Return के केस में answer को function के बहार भेजते हैं जिसे कोई अन्य variable ग्रहण करता है और फिर उसको प्रिंट करता है ।

```
= RESTART: C:/Users/KV
Enter value of a : 12
Enter value of b : 23
The sum is : 35
```

User-defined Functions with multiple return values (Practical)

```
'''Program which return multiple values'''
```

```
def calc(a,b):  
    add=a+b  
    sub=a-b  
    mul=a*b  
    div=a/b  
    return add,sub,mul,div
```

```
x=int(input("Enter value of a : " ))  
y=int(input("Enter value of b : " ))  
p,q,r,s=calc(x,y) ←  
print("The sum is : ",p)  
print("The difference is : ",q)  
print("The multiplication is : ",r)  
print("The quotient is : ",s)
```

Python में function multiple values को भी return कर सकता है जिसमें वह returning values का एक sequence return करता है जिसे हम बाद में प्राप्त करके एक एक करके प्रिंट करा सकते हैं।

```
= RESTART: C:\Users\KVBBKSer  
Enter value of a : 14  
Enter value of b : 7  
The sum is : 21  
The difference is : 7  
The multiplication is : 98  
The quotient is : 2.0
```

User-defined Functions with multiple return values (Practical)

```
'''Program which return multiple values'''
```

```
def calc(a,b):  
    add=a+b  
    sub=a-b  
    mul=a*b  
    div=a/b  
    return add,sub,mul,div
```

```
x=int(input("Enter value of x : "))  
y=int(input("Enter value of y : "))
```

```
result = calc(x,y)  
print("The answers are : ")  
for i in result:  
    print(i)
```

पिछले प्रोग्राम को हम ऐसे भी लिख सकते हैं ।

```
= RESTART: C:\Users\K  
Enter value of x : 23  
Enter value of y : 4  
The answers are :  
27  
19  
92  
5.75
```

Functions में Parameters और Arguments

- जब हम किसी function का header लिखते हैं तो उसके parantheses अर्थात कोष्ठक '()' में दी जाने वाली एक या अधिक values को **Parameter** कहा जाता है ।
- ये वह values होती हैं जिन्हें function किसी कार्य के लिए प्रयोग करता है ।
- जबकि **argument** वह वैल्यू है जिसे function को कॉल करते समय कोष्ठक (parantheses) में दिया जाता है ।
- दूसरे शब्दों में कहें की function को invoke करने के लिए arguments दिए जाते हैं ।
- Formal parameter को parameter तथा actual parameter को argument कहा जा सकता है ।

Functions में Parameters और Arguments. . .

```
'''Program which return multiple values'''
```

```
def calc(a,b):  
    add=a+b  
    sub=a-b  
    mul=a*b  
    div=a/b  
    return add,sub,mul,div
```

इस प्रोग्राम में यह *parameters* हैं |

```
x=int(input("Enter value of x : " ))  
y=int(input("Enter value of y : " ))
```

```
result = calc(x,y)  
print("The answers are : ")  
for i in result:  
    print(i)
```

इस प्रोग्राम में यह *arguments* हैं |

Arguments के प्रकार

- पाइथन में 4 प्रकार के arguments मान्य हैं -
 1. Positional Arguments
 2. Default Arguments
 3. Keyword Arguments
 4. Variable Length Arguments

```
'''Program to add two numbers using function  
with argument and return statement  
'''
```

```
def add(a,b):  
    c=a+b  
    return c
```

```
x=int(input("Enter value of a : " ))  
y=int(input("Enter value of b : " ))  
z=add(x,y)  
print("The sum is : ",z)
```

```
= RESTART: C:/Users/KV  
Enter value of a : 12  
Enter value of b : 23  
The sum is : 35
```

1. Positional Arguments

- ये वह arguments होते हैं जिन्हें function में सही positional order में pass किया जाता है |
- अर्थात् यदि arguments की स्थिति(position) बदल दें तो उत्तर भी बदल जाता है |

```
>>> def subtract(a,b):  
        print(a-b)
```

```
>>> subtract(200,100)  
100
```

```
>>> subtract(100,200)  
-100
```

2. Default Arguments

- ये वह arguments होते हैं जिनके द्वारा function को default वैल्यू प्रदान की जा सकती है ।
- अर्थात् यदि arguments में कुछ भी न pass किया जाये तो वह अपने आप पहले से निर्धारित वैल्यू को ले लेगा ।

```
>>> def wish(name="Sanjeev") :  
        print("Hello ", name, "! Happy Holi!")  
  
>>> wish("Pavan")  
Hello Pavan ! Happy Holi!  
>>> wish()  
Hello Sanjeev ! Happy Holi!
```

ध्यान ये रखना है की default arguments को हमें non-default arguments के बाद define करना होता है ।

3. Keyword Arguments

- यदि किसी function में कई arguments हैं और हम उनके क्रम को बदलना चाहें तो keyword argument का प्रयोग करते हैं।
- इसके फायदे ये हैं की इसमें हमें arguments का क्रम याद रखने की ज़रूरत नहीं होती है।
- इसके लिए हमें values को जब भी pass करना हो तो parameter के नाम के साथ pass करेंगे | e.g.

```
>>> def wish(name,msg):  
    print("Hello! ",name,"! ",msg,"!")  
  
>>> wish(name="Ram",msg="Good Evening")  
Hello!  Ram !  Good Evening !  
>>> wish(msg="Good Evening",name="Ram")  
Hello!  Ram !  Good Evening !  
>>> wish(name="Ram","Good Evening")  
SyntaxError: positional argument follows keyword argument  
>>> wish("Ram","Good Evening")  
Hello!  Ram !  Good Evening !
```

यहाँ देखने वाली बात ये है की अपने यदि एक keyword argument कॉल किया है तो बाकि भी keyword के द्वारा ही प्रयोग करना होगा | अन्यथा error आयेगी।

4. Variable Length Arguments

- जैसा की नाम से ही लग रहा है कि हम इसमें arguments की variable संख्या दे सकते हैं अर्थात ज़रूरत के अनुसार 1,2,3,4 . . . Arguments | ऐसे arguments को variable length arguments कहा जाता है |
- Variable length argument को देने के लिए asterisk (*) का प्रयोग करते हैं |

```
>>> def sum(*n):  
    s=0  
    for i in n:  
        s=s+i  
    print("The sum = ",s)  
  
>>> sum()  
The sum = 0  
>>> sum(10)  
The sum = 10  
>>> sum(10,20)  
The sum = 30  
>>> sum(10,20,30)  
The sum = 60
```

यहाँ देखने वाली बात ये है कि हर बार sum function को कॉल करने में अलग अलग संख्या में arguments दिए गए हैं | और दी गयी संख्या के अनुसार हमें उत्तर प्राप्त हो रहा है |

Function में ARRAYS /LISTS को pass करना

- पाइथन में हम list को ही array के रूप में प्रयोग करते हैं हालाँकि पाइथन में array के लिए हमें numpy module को import करना होता है |
- हम यहाँ list को ही function में pass करेंगे | array से बेहतर list होती हैं |

```
#Program to print the average of integers using list
```

```
def list_avg(lst):  
    l=len(lst)  
    sum=0  
    for i in lst:  
        sum+=i  
    return sum/l
```

```
a=input("Input Integers")  
a=a.split()  
for i in range(len(a)):  
    a[i]=int(a[i])  
  
avg=list_avg(a)  
  
print("Average is : ")  
print(round(avg,2))
```

```
RESTART: C:/Users/KVBBKServe.  
Input Integers1 3 4 5 6 7 8 9  
Average is :  
5.38
```

Variable का Scope

- Variable के scope का मतलब यह है की प्रोग्राम का वह हिस्सा जहाँ यह variable visible होगा अर्थात इसका उपयोग किया जा सकता है ।
- हम यह भी कह सकते हैं की scope, current variable और उनके मानों का समूह होता है ।
- Scope दो प्रकार का हो सकता है -
- Global (module)
 - वह सारे नाम जो module के टॉप लेवल में assign किये जाते हैं या सीधे interpreter में assign किये जाते हैं
- Local (function)
 - ये वह variable होते हैं जो किसी function या लूप के अन्दर declare किये जाते हैं ।

Using main() as a Function

- Python में main() आवश्यक नहीं है ।
- Non-python programmers की सुविधा के लिए हम इसका निम्नवत प्रयोग कर सकते हैं -

```
def hello():  
    print("Hello! World!")  
  
def main():  
    print("This is the main function")  
    hello()  
  
main()
```

```
= RESTART: C:/Users/KVBBKSe  
This is the main function  
Hello! World!
```

Function call के समय execution का flow

- किसी भी प्रोग्राम का Execution हमेशा पहली लाइन से शुरू होता है और एक एक लाइन करके execute करता हुआ चलता है ।
- एक बार में एक statement ही execute होता है।
- Function किसी भी प्रोग्राम के flow को बदलता नहीं है ।
- Function के अन्दर के statements तब तक execute होना शुरू नहीं होते जब तक कि function को कॉल न किया जाये ।
- जब function कॉल होता है तब प्रोग्राम के अगले स्टेटमेंट पर जाने के बजाय control function की body में jump कर जाता है ।
- फिर सारे function के statements एक एक करके top से bottom तक execute होते हैं ।
- और फिर से control उस स्थान पर आजाता है जहां पर function कॉल किया गया था ।
- इसी प्रकार क्रम से पाइथन का प्रोग्राम execute होता है ।

RSCURSION

- Recurision में function बार बार अपने आप को ही तब तक कॉल करता रहता है जब तक कि दी गयी condition संतुष्ट न हो जाये ।

```
# to calculate power of an inputed number using recursion
```

```
def power(x,n):  
    if n==0:  
        return 1  
    else:  
        return x*power(x,n-1)  
  
a=int(input("Enter the value of x : "))  
n=int(input("Enter the value of n : "))  
  
print("The result is : ",power(a,n))
```

```
= RESTART: C:/Users/KVBB  
Enter the value of x : 2  
Enter the value of n : 3  
The result is : 8
```

RSCURSION. . .

- Recurision, प्रोग्रामिंग भाषा के सबसे powerful tools में से एक है | किसी function के अपने आपको ही बार बार कॉल करने को ही recursion कहा जाता है |
- Recursion को प्रयोग करने की निम्न शर्तें हैं -
 - एक terminating condition होनी चाहिए |
 - Recursive routine में एक if condition होनी चाहिये |
 - प्रत्येक recursive problem का एक खास feature होता है उसका execution के order में reversal.
 - जितनी बार recursive कॉल होती है उतनी बार उन लोकल variable के लिए नयी memory allocate होती है जिनका प्रयोग recursion के लिए हो रहा है |
 - Recursion के दौरान प्रत्येक वैल्यू को एक stack में push कर दिया जाता है

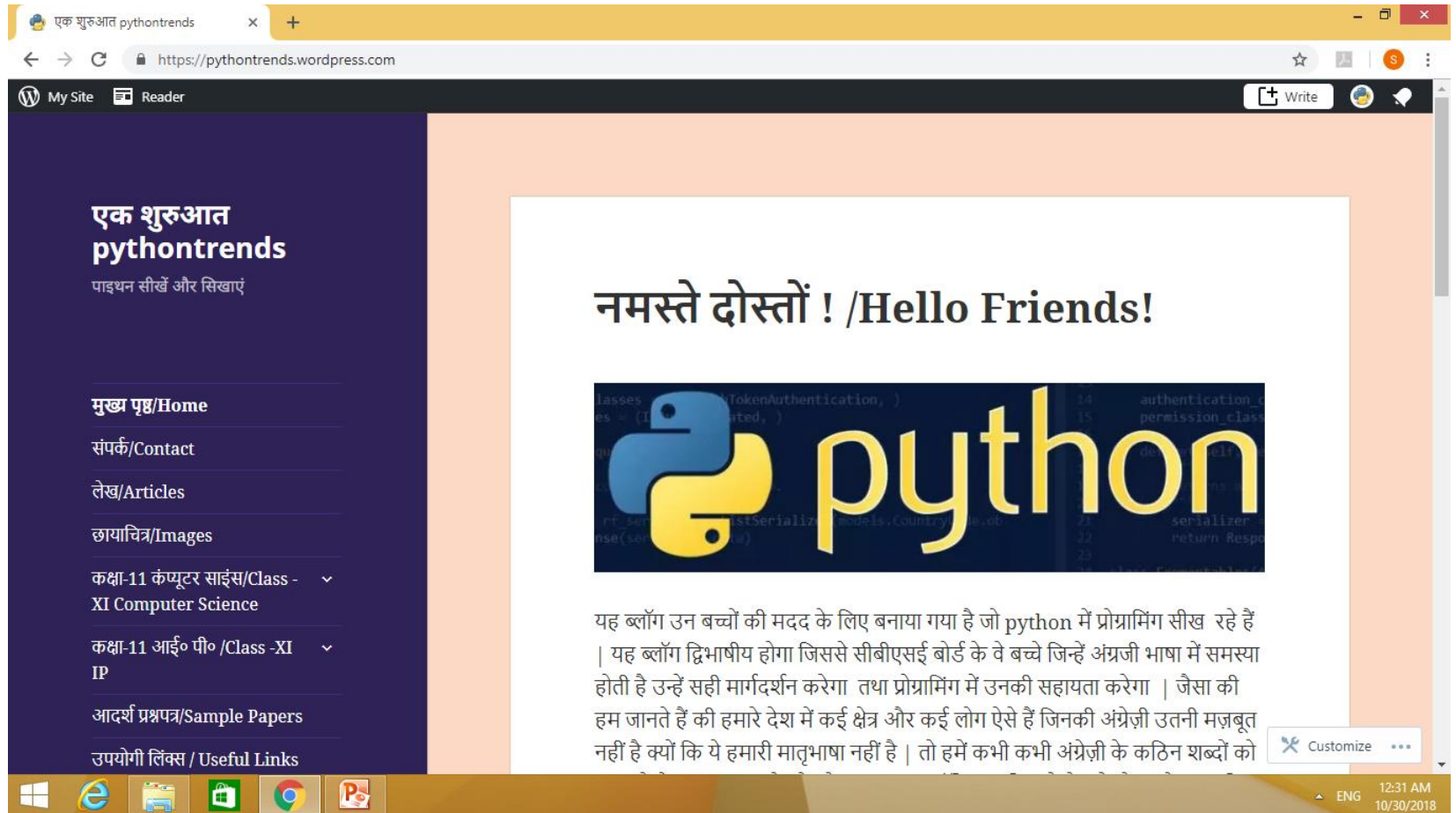
RSCURSION के नुकसान

- यह स्टोरेज ज्यादा प्रयोग करता है क्योंकि हर वैल्यू को stack में insert करता है ।
- अगर recursive calls को सही से चेक नहीं किया गया है तो आपका कंप्यूटर out of memory जा सकता है ।
- यह speed और execution time के लिहाज़ से कम उपयोगी होता है ।

धन्यवाद

और अधिक पाठ्य-सामग्री हेतु निम्न लिंक पर क्लिक करें -

www.pythontrends.wordpress.com



एक शुरुआत
pythontrends
पाइथन सीखें और सिखाएं

मुख्य पृष्ठ/Home
संपर्क/Contact
लेख/Articles
छायाचित्र/Images
कक्षा-11 कंप्यूटर साइंस/Class -
XI Computer Science
कक्षा-11 आई० पी० /Class -XI
IP
आदर्श प्रश्नपत्र/Sample Papers
उपयोगी लिंक्स / Useful Links

नमस्ते दोस्तों ! /Hello Friends!



यह ब्लॉग उन बच्चों की मदद के लिए बनाया गया है जो python में प्रोग्रामिंग सीख रहे हैं | यह ब्लॉग द्विभाषीय होगा जिससे सीबीएसई बोर्ड के वे बच्चे जिन्हें अंग्रेजी भाषा में समस्या होती है उन्हें सही मार्गदर्शन करेगा तथा प्रोग्रामिंग में उनकी सहायता करेगा | जैसा की हम जानते हैं की हमारे देश में कई क्षेत्र और कई लोग ऐसे हैं जिनकी अंग्रेजी उतनी मज़बूत नहीं है क्यों कि ये हमारी मातृभाषा नहीं है | तो हमें कभी कभी अंग्रेजी के कठिन शब्दों को

Customize ...

ENG 12:31 AM 10/30/2018